

Informe Técnico: Desarrollo de un Sistema de Recomendación Musical

1. Introducción y Contexto del Proyecto

El presente informe documenta el desarrollo, análisis y evaluación de diversos modelos de Filtrado Colaborativo para un Sistema de Recomendación Musical. El proyecto aborda el reto de construir recomendaciones precisas partiendo de datos de **feedback implícito** (historial de escuchas o *playcounts*).

El objetivo principal ha sido contrastar arquitecturas clásicas de vecindario y factorización matricial frente a aproximaciones de aprendizaje profundo (Deep Learning), evaluando no solo el error de predicción, sino la calidad real del *ranking* o descubrimiento musical.

2. Preprocesamiento de Datos (EDA)

El conjunto de datos original nace de la intersección del **Million Song Dataset, Spotify y Last.fm**. El estado inicial del dataset presentaba 962,037 usuarios, 30,459 canciones y más de 9.7 millones de interacciones, con una densidad muy baja (0.033%).

Para preparar los datos para el modelado, se tomaron las siguientes decisiones arquitectónicas:

1. **Filtrado de Ruido (K-Core Filtering):** Se eliminaron usuarios con menos de 5 escuchas y canciones con menos de 10 reproducciones.
2. **Transformación del Target (Playcounts a Ratings):** Los *playcounts* (rango original de 1 a 2,948) se transformaron a un rating sintético continuo. Esto se logró aplicando una función logarítmica ($\log_{1p}$), recorte de valores atípicos al percentil 99 y un escalado *min-max*. Esta decisión permitió homogenizar el espacio de error (MSE) para compararlo con arquitecturas estándar de la literatura.
3. **División de Datos (Train/Test):** Se aplicó un *split* estratificado 80/20 por usuario, garantizando la presencia (problema *warm-start*) de todos los usuarios en ambos conjuntos.
4. **Optimización de Memoria:** Las matrices finales de interacción se almacenaron en formato de matriz dispersa (`scipy.sparse.csr_matrix`) y se serializaron en archivos `.npz`, asegurando alta eficiencia en consumo de RAM.

Resumen del Dataset Final:

- **Usuarios Únicos:** 533,689
- **Canciones:** ~25,000
- **Interacciones:** 8,650,751 (se conservó el 89% de la información original)

3. Arquitecturas y Modelos Evaluados

Se experimentó con cuatro familias de algoritmos distintos, cada uno aportando una perspectiva diferente al problema:

3.1. K-Nearest Neighbours (KNN)

Modelo clásico basado en memoria (vecindario).

- **Hallazgos:** Sufre de un fuerte **s sesgo de centralidad**. Al promediar desviaciones, el modelo suaviza las predicciones hacia la media general (MAE estancado en ~1.2 estrellas), impidiéndole predecir valoraciones extremas ("1" o "5" puros).
- **Perfil de uso:** Actúa como un excelente **Motor de Descubrimiento**. Aunque su Precisión exacta es baja (8%), posee un alto Recall (45%) y un sólido NDCG (44%), lo que significa que posiciona contenido altamente relevante en la cima de la lista de recomendaciones.

3.2. Probabilistic Matrix Factorization (PMF)

Implementado bajo el paradigma de *Neural Collaborative Filtering*.

- **Mecánica:** Basa su predicción de forma estricta en el producto escalar puro de factores latentes de usuario e ítem: $\hat{r}_{u,i} = \sigma(\mathbf{p}_u^T \mathbf{q}_i) \cdot 4.0 + 1.0$.
- **Evaluación:** Se aplicó una metodología rigurosa utilizando *Negative Sampling* en la fase de test, garantizando que el modelo fuera evaluado con exposición cero a dichas muestras durante el entrenamiento (optimizado por lotes mediante PyTorch).

3.3. Bernoulli Matrix Factorization (BeMF)

Modelo probabilístico que divide el problema en 5 sub-modelos logísticos (uno por cada estrella del 1 al 5).

- **Optimizaciones:** Implementado con SGD acelerado en Numba, incluyendo términos de *bías*, inicialización basada en *log-odds* y *ordinal label smoothing*.
- **Hallazgos:** A pesar de ser matemáticamente sofisticado, **no logró superar al modelo de referencia (Baseline Bias-Only)**. Esto se debe a que BeMF fue diseñado para *ratings explícitos discretos* (ej. MovieLens), y sus supuestos estadísticos colapsaron al enfrentarse a una variable continua sintética derivada de *playcounts*.

3.4. Multi-Layer Perceptron (MLP / NCF)

Enfoque de *Deep Learning* puro.

- **Arquitectura:** Concatena los *embeddings* densos de usuarios e ítems (a diferencia del producto Hadamard de GMF) y los pasa por múltiples capas densas no lineales con funciones de activación ReLU.
- **Hallazgos:** Capaz de capturar relaciones complejas no lineales que escapan a los modelos de producto escalar simple. Su fortaleza reside en optimizar el *ranking* general de

la lista propuesta en lugar de minimizar el error de regresión numérico.

4. Conclusiones Principales

1. **La naturaleza del dato dicta el rendimiento del modelo:** El experimento con BeMF demuestra empíricamente que la complejidad matemática de una arquitectura no garantiza mejoras si sus supuestos fundacionales (datos discretos/clases balanceadas) no encajan con la naturaleza real de los datos introducidos (un continuo sintético derivado de feedback implícito).
2. **El dilema de las métricas (Descubrimiento vs. Regresión):** Un sistema puede tener un pobre desempeño pronosticando la calificación numérica exacta (MAE/RMSE) y, sin embargo, ser un producto de software excelente en producción gracias a su capacidad de colocar canciones que el usuario amará al inicio de su pantalla (alto NDCG).
3. **Eficiencia Computacional:** Manejar más de 8 millones de interacciones validó la necesidad absoluta de utilizar ecosistemas paralelos (PyTorch/GPU) y estructuras de datos eficientes (csr_matrix y Numba JIT) en el proceso de optimización iterativa.

5. Trabajo Futuro Recomendado

A partir de los hallazgos de este estudio, las líneas de evolución natural para el sistema son:

- **NeuMF (Neural Matrix Factorization):** Fusionar el modelo GMF (General Matrix Factorization) y el MLP desarrollado en una arquitectura paralela (siguiendo a *He et al.*), lo cual históricamente ha demostrado superar a ambas ramas por separado.
- **Reformulación Binaria:** Transformar el enfoque de regresión continua a uno de clasificación binaria estricta (escuchó vs. no escuchó) utilizando *Negative Sampling* masivo en entrenamiento, acercando el modelo a la verdadera naturaleza del *feedback implícito*.
- **Regularización:** Implementar *Dropout* en las capas del MLP para permitir el aumento de la dimensionalidad de los vectores latentes sin incurrir en *overfitting*.